

Oceanpoint Whitepaper

A Liquidity Engine for Tokenized Real Estate

Unifying tokenized real estate assets through the use of a dynamic liquidity aggregator token.

Authors: Denis Petrovcic, Daniel Kaines

Contributors: Simon Kruise, David Šenica, Viktor Brajak, Marko Majetić, Kristjan Bajuk

April 2025

Table of Contents

1. Executive Summary	4
2. Oceanpoint's Evolution: Laying the Foundations	5
2.1. Governance Pool: The Genesis of Stakeholder Involvement	5
2.1.1. Main Pool (sBST)	5
2.1.2. Marketplace Pools (Launchpad)	5
2.2. Asset Pools: Unlocking Yield for Tokenized Real Estate	6
2.2.1. Issuer BSPT Pool	6
2.2.2. Community BSPT Pool	6
2.2.3. Marketplace BSPT Pools	7
2.3. Liquidity Pools: Bootstrapping Ecosystem Liquidity	7
3. Problem Statement: Liquidity & Access in Tokenized Real Estate	8
4. Oceanpoint: A Liquidity Engine for Blocksquare's Ecosystem	9
4.1. Tokenization: The Liquidity Challenge	9
4.2. POINT: A Liquidity Aggregator Token	9
4.2.1. Can POINT Acquire Flatcoin-Like Qualities?	10
4.2.2. POINT Overcollateralization and Ratio	10
4.3. POINT: Expanding BST's Utility & Influence	11
The Expanding Role of BST	12
5. Multi-Phased Roadmap	12
5.1. Phase I: Bootstrapping Liquidity (Stablecoin ↔ POINT)	12
5.1.1. Liquidity Staking	12
5.2. Phase II: BSPT Integration	13
5.2.1. Dynamic Rate System and Target Portfolio Composition	13
NAV Discount Rate	13
NAV Premium Rate	13
Stablecoin Deposit Tax	13
Stablecoin Withdrawal Tax	13
5.2.2. Marketplaces and instant swaps	14
5.3. Phase III: Advanced Risk Management	14
5.3.1. POINT Zero: The Protocol's Stability Backstop	15
5.4. Phase IV: BSPT Lending	16
6. System Architecture & Technical Design	16
6.1. Core Contracts & Upgradeability	17
6.1.1. OceanpointToken (POINT)	17
6.1.2. OceanpointProtocol	18

6.1.3. OceanpointRouter (OPR)	28
Acquiring BSPTs	29
Selling BSPTs	29
Acquiring POINT Tokens	29
Selling POINT Tokens	29
6.2. Buying/Selling BSPTs Flowcharts	32
6.3. Example Flows (Oceanpoint)	33
6.3.1. Deposit Stablecoins → Mint POINT	33
6.3.2. Burn POINT → Withdraw Stablecoins	33
6.3.3. Sell BSPTs → Receive POINT (Mint)	34
6.3.4. Pay POINT → Buy BSPTs (Burn)	35
6.4. Example Flows (Marketplaces)	36
6.4.1. Sell Stablecoins → Get BSPTs	36
6.4.2. Sell BSPTs → Get Stablecoins	37
7. Risk Management & Governance	37
7.1. Dynamic Rate Adjustments	38
7.2. Overcollateralization & In-protocol Exchange Rate	39
7.3. Managing Undercollateralization Scenarios	39
7.3.1. Mitigation 1: Increasing Spreads	39
7.3.2. Mitigation 2: Active Management of BSPT Purchase Discounts	40
7.4. Governance Roadmap	40
8. Simulations & Stress Testing	41
8.1.1. Simulation Design and Purpose	41
8.1.2. Test Phase 1: Baseline Simulations	41
8.1.3. Test Phase 2: Stress Testing	42
8.1.4. Test Phase 3: Multi-Event Stress Simulation	44
8.1.5. Summary	45
9. Conclusion & Outlook	46

1. Executive Summary

Oceanpoint is Blocksquare's core defi protocol designed to unify and enhance liquidity across Blocksquare's ecosystem of tokenized real estate. Blocksquare Property Tokens (BSPTs) unlock fractionalized ownership of real estate on-chain, yet they often face limited liquidity, market fragmentation, and require KYC verification to be accessible for users to hold. Oceanpoint aims to solve these challenges by acting as a central liquidity engine for real estate tokens, providing a simple gateway for users to move between stablecoins, property tokens, and other digital assets.

Within this framework, two tokens occupy center stage: BST and POINT.

BST has historically served as the governance and utility token of Blocksquare, rewarding liquidity providers, granting holders a voice in protocol decisions, offering discounts on software licensing and usage, and ultimately holding the potential of capturing the network's growth. POINT, by contrast, is introduced as an aggregator token that simplifies how BSPTs (Blocksquare Property Tokens) and stablecoins interact, effectively merging real estate assets into the broader realm of decentralized finance (DeFi).

Mintable through the deposit of stablecoins and eventually BSPTs, POINT serves as an aggregator token for tokenized real estate properties. This aggregation mechanism allows BSPT investors to access liquidity more quickly than they would if they were seeking to enter or exit through the secondary market for individual BSPTs. POINT's creation elevates the Blocksquare ecosystem by upgrading Oceanpoint into a scalable liquidity engine that, in turn, increases the reach and utility of BST. Specifically, as more properties and stablecoins flow into Oceanpoint, BST holders and stakers stand to benefit from deeper liquidity, additional reward opportunities, and an expanding user base. Over time, the synergy between these two tokens—BST's governance/utility roles and POINT's aggregator functionality—will enhance the ecosystem's resilience and accelerate the adoption of tokenized real estate.

Since the launch of Oceanpoint v0.1, Blocksquare has implemented various protocols and staking mechanisms that proved instrumental for understanding user behavior and liquidity management. These developments also underscored the indispensable role of BST in coordinating and rewarding ecosystem participants.

The rollout of POINT will involve multiple phases, commencing with Phase I, which will enable stablecoin swaps to establish circulating supply and liquidity. Phase II will introduce BSPTs swaps to incorporate real estate exposure. Subsequent phases, III and IV, will introduce advanced risk management and lending features. The overarching aim is to develop a holistic DeFi ecosystem that merges digital finance and real estate.

2. Oceanpoint's Evolution: Laying the Foundations

The strategic development stages that Oceanpoint underwent were essential in establishing its role as the liquidity engine for tokenized real estate. These early iterations were instrumental in introducing core functionalities and providing valuable insights into market behavior, user engagement, and operational dynamics.

2.1. Governance Pool: The Genesis of Stakeholder Involvement

Governance Pools represent one of the most significant advancements during this period, laying the groundwork for stakeholder involvement and long-term alignment within Blocksquare's ecosystem. These pools not only fostered early community participation but also laid the groundwork for decentralized decision-making. It provided essential learnings about user incentives and the importance of aligning rewards with long-term protocol engagement.

2.1.1. Main Pool (sBST)

The Main Pool was introduced as a straightforward mechanism for users to deposit their BST tokens and earn rewards in the form of sBST, a governance token. This pool established a direct link between participants and the decision-making processes within Oceanpoint. By staking their tokens, users could actively support the platform's development and participate in governance through voting, while earning returns proportionate to their contributions.

2.1.2. Marketplace Pools (Launchpad)

A notable extension of this phase was the launch of Marketplace Pools (The Marketplace Launchpad), a mechanism designed to enable Certified Partners (CPs) to gather community support for establishing their tokenized real estate marketplaces. Through these pools, CPs can present their business plans and KPIs to the community, seeking pledges of sBST tokens to help crowdsource the software license fee (100,000 sBST) required to launch their platforms. This initiative not only decentralized access to marketplace creation but also reinforced community engagement and accountability.

2.2. Asset Pools: Unlocking Yield for Tokenized Real Estate

Asset Pools form a central pillar of Oceanpoint's staking features, designed to create a first bridge between tokenized real estate and decentralized finance (DeFi). By enabling property owners and holders of BSPTs to actively stake tokenized real estate, these pools create a dynamic mechanism to encourage early participation by offering attractive yields.

To date, three distinct features within this category have been launched:

1. Issuer BSPT Pool
2. Community BSPT Pool
3. Marketplace BSPT Pools.

In summary, the introduction of Asset Pools represents a strategic move to unlock the potential of tokenized real estate. Oceanpoint has created a path towards a real estate ownership model that is more participatory, distributed, and independent by bridging the gap between traditional real estate stakeholders and the decentralized finance (defi) ecosystem.

2.2.1. Issuer BSPT Pool

The BSPT Issuer Pool offers property owners who are also interested in crypto and defi a unique incentive to tokenize their real estate assets using Blocksquare's tokenization infrastructure and issue their own unique BSPT tokens. By issuing and subsequently staking 10,000 BSPT, which represents 10% of their real estate property's economic value, issuers can earn BST rewards and contribute to the overall growth of tokenized value on the Blocksquare ecosystem.

The Issuer BSPT Pool provides an opportunity for landlords to earn passive income from their tokenized assets and aligns their interests with Blocksquare's growth over the long term. It also serves as a vital gateway for traditional real estate stakeholders to enter the DeFi space through a learn-to-earn model, showcasing the ease of integrating tokenization into property owners' financial management and investment strategies.

2.2.2. Community BSPT Pool

The Community BSPT Pool, introduced in v0.4, enhanced asset pool functionality by allowing community members who hold BSPT, but are not issuers, to stake tokenized real estate. By staking BSPT, users effectively redirect regular rental revenues toward the DAO treasury and in exchange receive BST rewards — a tradeoff and risk compensated with a higher APY.

More importantly, staking BSPTs in this pool plays a key role in the deflationary mechanics of BST tokenomics. The DAO treasury utilizes rental revenues to buy back and burn BST tokens, which reduces the overall circulating supply and increases long-term token scarcity. As the scale of tokenized properties and rental revenues grow, this deflationary effect could potentially strengthen the ecosystem's economic foundation and create value for long-term holders.

2.2.3. Marketplace BSPT Pools

The Marketplace BSPT Pool, which evolved from the Community BSPT Pool concept, applies similar mechanisms to properties listed on Certified Partner (CP) marketplaces. This type of asset pool allows CPs to boost revenue for tokenized properties on their marketplace. Stakers receive BST rewards in exchange for their participation, and the rental income from the tokenized properties is directed to the DAO treasury.

These pools operate as a time-limited boosting service for marketplace operators. The service allows CPs to incentivize user participation by providing the unique value proposition of "hold real estate, earn crypto," which seamlessly combines real-world assets with web3 rewards.

2.3. Liquidity Pools: Bootstrapping Ecosystem Liquidity

The introduction of Liquidity Pools in Oceanpoint v0.3 was a fundamental step in connecting tokenized real estate with the decentralized finance (DeFi) ecosystem. These pools allow users to earn additional yield through Oceanpoint when they provide liquidity on decentralized AMM exchanges like Uniswap.

The mechanism applies to target asset pairs, with BST:ETH being the first one to be introduced. Participants who add liquidity to this pool on Uniswap receive Liquidity Provider (LP) tokens, representing their share in the AMM pool, which can then be staked on Oceanpoint to earn BST rewards. This mechanism not only boosts liquidity for target crypto assets, but also offers users an avenue for offsetting potential effects of impermanent loss.

The liquidity provision mechanism has initially been applied to the BST:ETH asset pair on Uniswap. Liquidity providers (LPs) who add liquidity to this specific pool receive LP tokens, representing their share in the AMM pool. These LP tokens can then be staked on the Oceanpoint platform to earn BST rewards. Additionally, stakers are given the choice of making a commitment to lock up their liquidity for longer timeframes, which gives them a boost in their staking pool share. This mechanism not only boosts liquidity for target crypto assets, but also offers users an avenue for offsetting potential effects of impermanent loss.

Unlike traditional liquidity mining campaigns in DeFi—often defined by fixed rewards and limited timeframes—Oceanpoint’s Liquidity Pools are designed for perpetual operation. As of the time of writing, the rewards come from an allocation of BST tokens put aside for staking rewards. But in time, and as the ecosystem grows, rewards shall come by leveraging the continuous cash flows generated by real estate assets, such as rental revenues from lease agreements, these pools ensure a steady source of rewards.

Finally, the smart contract logic for these pools can be adapted to support various asset pairs beyond the initial BST:ETH pairing. For example, liquidity pools could be created for pairs such as BST:USDC, BST:USDT, or BST:POINT, allowing participants to provide liquidity and earn rewards through staking. By expanding the range of asset pairs, Oceanpoint can offer risk diversification for community members while maintaining liquidity in various market conditions.

3. Problem Statement: Liquidity & Access in Tokenized Real Estate

The current environment for tokenized real estate, particularly BSPTs, presents a set of interlinked challenges. Each property is fractionalized into a unique BSPT token with a fairly small or even micro market cap. This fragmentation results in low liquidity and high spreads, making it difficult for users to quickly sell or trade BSPT tokens. Furthermore, access to any property token is subject to KYC requirements, creating an additional barrier for investors to engage. While these constraints aim to meet regulatory standards, they deter potential buyers who crave the simplicity and speed typical of broader crypto markets.

On the other side, stablecoins, while liquid, offered no direct upside from real estate’s potential growth. Meanwhile, tokenized real estate marketplaces lack deeper liquidity and struggle to deliver a seamless trading experience to their user base. The result is that many participants—ranging from smaller investors to institutional players—remain hesitant to fully embrace tokenized real estate, knowing that cashing out quickly could be challenging.

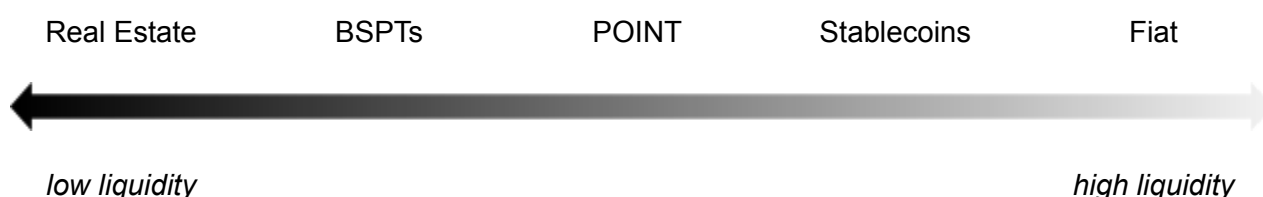
Recognizing these structural limitations, Blocksquare conceived Oceanpoint as a specialized liquidity engine to unify property tokens and stablecoins under a single aggregator token—POINT—without compromising BST’s established governance and utility value.

4. Oceanpoint: A Liquidity Engine for Blocksquare's Ecosystem

Oceanpoint is designed to serve as a unifying protocol for tokenized real estate, integrating stablecoins, BSPTs, and DeFi protocols under one cohesive system. By doing so, it dramatically simplifies the process of swapping between real estate tokens and other digital assets, effectively reducing friction and enhancing liquidity in the broader ecosystem.

4.1. Tokenization: The Liquidity Challenge

Tokenizing real estate via BSPTs (Blocksquare Property Tokens) represents a major leap toward fractional ownership of an inherently illiquid asset class — real estate. However, BSPTs still sit relatively low on the liquidity spectrum, unlike the almost instant settlement and global 24/7 trading that is characteristic of decentralized finance (DeFi).



Real estate's physical nature and high value make it difficult to trade, placing it on the far left of the liquidity spectrum as a highly illiquid asset. Tokenization into BSPTs improves the liquidity of real estate, but it still doesn't compare to the almost frictionless trading environment of DeFi, resulting in slow adoption within the web3 investor community. Stablecoins like USDT or USDC, which are redeemable globally and usually backed 1:1 by fiat or an equivalent reserve, are much more liquid. Finally, at the far right of the liquidity spectrum sit fiat currencies like USD, which, while not 100% frictionless, can be considered highly liquid relative to real estate.

4.2. POINT: A Liquidity Aggregator Token

POINT, an ERC-20 token with unlimited supply, can only be minted or burned by depositing or withdrawing specific stablecoins or eligible BSPTs. Designed to bridge liquidity between stablecoins and tokenized real estate properties, the Oceanpoint protocol always enables users to redeem one POINT for 1\$ value in stablecoins or BSPTs.

Oceanpoint bridges liquidity gaps by positioning its aggregator token POINT between BSPTs and the broader DeFi ecosystem. To sell a specific BSPT, users can exchange it for POINT tokens, which Oceanpoint then swaps through external DEX liquidity (e.g., a Uniswap pool) for near-immediate settlement into the desired stablecoin (e.g. USDC). Conversely, to buy tokenized real estate, users can spend e.g. USDC, which Oceanpoint routes into POINT and then redeems for the target BSPT.

4.2.1. Can POINT Acquire Flatcoin-Like Qualities?

Although POINT tokens can be redeemed at a 1:1 ratio, which may initially appear to categorize them as stablecoins, they differ significantly. While stablecoins, like USDC, traditionally strive for a 1:1 peg to a fiat currency, POINT is an aggregator token that can potentially incorporate an expanding basket of stablecoins and real-estate-backed tokens. This approach moves POINT away from the strict definition of a stablecoin into territory often referred to as “flatcoins.”

Flatcoins are a new type of cryptocurrency designed to maintain purchasing power over time, even during periods of inflation. Unlike stablecoins, which are pegged to a stable asset, flatcoins are linked to economic indicators or indices, such as the cost of living. This design helps ensure that their value remains stable in terms of real-world purchasing power, even as inflation occurs. Flatcoins utilize pegging to inflation-adjusted indices, meaning that their value can increase in line with inflation rates, theoretically preserving the purchasing power of the currency over time. This approach is more complex than traditional stablecoin mechanisms and relies on accurate economic data for effectiveness.

As Oceanpoint develops, POINT may become less aligned with a single USD peg and more aligned with a diverse selection of real estate assets and a reserve of stablecoins. This would allow POINT to track the overall economic value of real estate and maintain stability without being solely algorithmic or fiat-collateralized.

4.2.2. POINT Overcollateralization and Ratio

From the outset, users deposit stablecoins (e.g. USDC) at a simple 1:1 ratio to mint POINT. Over time, as the treasury accumulates more BSPT tokens and stablecoin reserves, the system may become overcollateralized. This surplus coverage may provide the basis for altering the 1:1 ratio slightly in favor of POINT holders. In other words, you may eventually receive more stablecoins or BSPTs per 1 POINT if the protocol’s treasury collateral value consistently exceeds its circulating supply.

Unlike a typical stablecoin that attempts to lock its price or supply via strict pegging or algorithmic rebalancing, POINT could slowly and over a longer period of time track the growth and

diversification of real-estate value. Such an approach harnesses real estate's generally inflation-hedged nature and merges it with DeFi's liquidity advantages—acquiring “flatcoin-like” qualities driven by protocol overcollateralization.

4.3. POINT: Expanding BST's Utility & Influence

One of the most common questions in the Blocksquare community is how BST and POINT interact. Rather than competing, these two tokens work in synergy, each playing a distinct and complementary role within the ecosystem.

- BST is the ecosystem's utility token:
 - BST holders can earn yields by providing various services that add value to the protocol; staking liquidity, staking for governance.
 - BST holders benefit from the protocol's growth via token buybacks that reflect economic activity in the ecosystem. Depending on governance, these buybacks are used to execute either deflationary token burns or reward distributions to active participants.
 - Staking BST grants holders sBST, which empowers them to influence the evolution of the Blocksquare ecosystem and govern key Oceanpoint parameters, including Dynamic Rate System, Target Portfolio Composition, Target BSPT Ratio, BSPT Eligibility Criteria, Supported Stablecoin Integrations. As Oceanpoint grows, so does BST's governance authority.
- POINT is the ecosystem's liquidity aggregator token:
 - POINT provides seamless liquidity for tokenized real estate, acting as a universal bridge between BSPTs and stablecoins.
 - As the adoption of tokenized real estate expands, POINT drives increased transaction volumes, generating new liquidity flows that enhance BST's role within the ecosystem.
 - Most swaps between BSPTs and stablecoins will be routed through the BST:POINT liquidity pool, reinforcing BST's economic relevance by creating continuous demand for staking, governance participation, and liquidity incentives.

The Expanding Role of BST

BST holders directly benefit from POINT's adoption. As the aggregator token sees wider usage, it amplifies transaction activity, strengthening BST's position as the economic backbone of Oceanpoint. Increased liquidity flows translate into greater governance influence, deeper market participation, and sustained demand for BST-based staking and reward mechanisms.

With POINT fueling seamless real estate liquidity, we believe BST has the potential to become an even more powerful governance asset, ensuring those who stake and participate in decision-making remain at the core of Blocksquare's long-term success.

5. Multi-Phased Roadmap

Oceanpoint's roadmap consists of four major phases, each expanding the protocol's capabilities in a secure, compliant, and scalable manner. While the end goal is a seamless real estate liquidity engine, each phase is designed to introduce additional functionality in a measured and secure way, while preserving BST's core role within the protocol and wider ecosystem.

5.1. Phase I: Bootstrapping Liquidity (Stablecoin ↔ POINT)

The initial phase will establish the foundational core logic for minting and burning POINT tokens. This process will always be executed at a fixed 1:1 ratio between POINT and stablecoins (DAI, USDC, USDT). Users can deposit stablecoins and receive an equivalent amount of POINT tokens. Conversely, they can burn POINT tokens to withdraw stablecoins from Oceanpoint. This phase's focus on stablecoins will establish a baseline level of liquidity and a clear TVL metric.

5.1.1. Liquidity Staking

The initial phase will involve the launch of a new BST:POINT liquidity pool on Uniswap v2. The decision to utilize Uniswap v2 is based on its user-friendly interface and compatibility with our existing smart contract for staking LP tokens. This contract has been developed, audited, and extensively tested since v0.3 and incorporates boosting mechanisms that our community is already accustomed to, designed to promote long-term liquidity. Rewards for this pool will be sourced from the total monthly allocation designated for liquidity pools. This new pool will compete with other Oceanpoint pools within the Liquidity Pools category, such as BST:ETH.

Key Highlights:

- **Mint/Burn:** 1:1 ratio with stablecoins (e.g., USDC, DAI).
- **Liquidity Staking:** Add liquidity on Uniswap to earn BST rewards on Oceanpoint.
- **Objective:** Achieve an initial supply of POINT, set the foundation for future BSPT integration.

5.2. Phase II: BSPT Integration

Building upon the stablecoin foundation established in Phase I, Phase II will introduce support for BSPTs (Blocksquare Property Tokens). This expansion allows BSPT holders to sell their tokens into the Oceanpoint protocol in exchange for POINT tokens and vice versa, for users to buy BSPT tokens from Oceanpoint. This facilitates a crucial bridge between the traditionally illiquid real estate market, the emerging tokenized real estate market (represented by BSPTs) and the enhanced liquidity of the POINT aggregator token. The existing 1:1 redemption mechanism for stablecoins will be maintained, ensuring POINT holders can seamlessly redeem their POINTs for stablecoins at any given time.

5.2.1. Dynamic Rate System and Target Portfolio Composition

Phase II is also set to introduce a **Dynamic Rate System** built into the smart contract logic. Coupled with the **Target Portfolio Composition**, the smart contract will be able to make automatic gradual adjustments to guide the market towards the protocol's **Target BSPT Ratio**. This mechanism is designed to prevent an overconcentration of any single asset category within the protocol, ensuring balance of BSPT and stablecoins in the portfolio.

NAV Discount Rate The percentage discount applied when the treasury buys BSPTs from users. A higher discount encourages BSPT purchases at lower prices, helping rebalance the treasury when BSPTs are overweight.	NAV Premium Rate The percentage premium applied when the treasury sells BSPTs to users. A higher premium makes BSPTs more expensive, discouraging excessive purchases when the treasury is underweight in stablecoins.
Stablecoin Deposit Tax A fee charged when users deposit stablecoins into the treasury. Higher deposit taxes discourage inflows when the treasury has excess stablecoins.	Stablecoin Withdrawal Tax A fee charged when users withdraw stablecoins from the treasury. A higher withdrawal tax discourages outflows, ensuring the protocol retains enough liquidity to maintain stability.

For Phase II, the DAO's Governance Board will set the **Target BSPT Ratio** to be initially set at 20%, while the below parameters for the dynamic rate system will be adopted.

- **NAV Discount Rate:** Within a minimum of 0% and a maximum of 20%.
- **NAV Premium Rate:** Within a minimum of 0% and a maximum of 5%.
- **Stablecoin Deposit Tax:** Within a minimum of 0% and a maximum of 2%.
- **Stablecoin Withdrawal Tax:** Within a minimum of 5% and a maximum of 15%.

5.2.2. Marketplaces and instant swaps

In Phase II, real-estate owners can deposit BSPT tokens directly in exchange for POINT tokens, providing them with immediate liquidity. This process adds a real-estate dimension to the aggregator token. Users can swap BSPT for DAI seamlessly through a two-step process: BSPT to POINT, and then POINT to DAI. Investors can also convert DAI to BSPT in a similar, streamlined manner: DAI to POINT, and then POINT to BSPT.

To enhance the user experience and streamline the multi-step processes involved in swapping between BSPT, POINT, and stablecoins, a specialized Oceanpoint Router will be implemented. This router will be integrated directly into the marketplace platform's UI, enabling seamless swaps while abstracting the swap routing from the user. This positions the POINT token as both a liquid asset and a unique feature for marketplace operators within the ecosystem.

Key Highlights:

- **Deposit BSPTs:** Mint POINT based on property valuation.
- **Redemptions:** Burn POINT tokens to redeem stablecoins or BSPTs.
- **Dynamic Rate System:** NAV Rates and Stablecoin Taxes dynamically shift, encouraging the treasury to self-balance and reach the target BSPT ratio.
- **Oceanpoint Router:** Streamline multi-step swaps between BSPT, POINT, and stablecoins into single transactions.
- **Objective:** Enhance liquidity for specific property tokens and increase the protocol's portfolio exposure to cash-flowing real estate assets.

5.3. Phase III: Advanced Risk Management

Phase III's objective is to increase the protocol composition from Phase II's 20% BSPT to 80% BSPT. As BSPTs and stablecoins accumulate, Phase III will introduce an advanced system for managing risk, collateral ratios, and BSPT thresholds. Building on Phase II's foundation, discounts and premiums will adjust NAV Premium Rates or Stablecoin Redemption Taxes based on whether

the protocol is overweight in real estate or stablecoins. In Phase III a more refined strategy in setting up the min/max values shall be put in place by the Governance Board.

Over time, if the system becomes overcollateralized, POINT holders may benefit from improved redemption ratios. Conversely, if collateral falls below safe thresholds, the protocol can execute a rebalancing mechanism.

To mitigate the risk of undercollateralization, Phase III formalizes the "POINT Zero" concept, a backstop strategy similar to buy-and-burn mechanics. In the event of a crisis or severe undercollateralization, the protocol can release stablecoins to purchase POINT on the open market, reducing the number of tokens in circulation until equilibrium is restored.

5.3.1. POINT Zero: The Protocol's Stability Backstop

As Oceanpoint transitions into Phase III, the protocol introduces POINT Zero, a strategic mechanism designed to safeguard the system from severe undercollateralization. This mechanism acts as a last-resort stabilization tool, ensuring that the protocol remains solvent and fully backed, even in extreme market conditions.

POINT Zero is designed to mitigate risks associated with collateral imbalances by implementing a buy-and-burn mechanism when the protocol's collateral ratio falls below governance-defined safety thresholds. The system monitors treasury composition, dynamically adjusting to maintain stability, but in cases where collateral deterioration threatens overall solvency, POINT Zero intervenes by:

1. **Assessing the Deficit** – The protocol detects when Total Collateral Value (USD value of all BSPTs + stablecoins) falls below the circulating supply of POINT tokens.
2. **Deploying Stablecoin Reserves** – A controlled release of stablecoins from the protocol is used to purchase POINT tokens from the open market.
3. **Executing a Buy & Burn Process** – The acquired POINT tokens are burned and permanently removed from circulation, reducing the total supply.
4. **Restoring Collateral Balance** – Step 3 recalibrates the system and returns it to the target collateralization levels, ensuring that Oceanpoint remains a sustainable liquidity engine.

By introducing POINT Zero, Phase III formalizes Oceanpoint's ability to self-correct under stress. This ensures that even in periods of extreme market volatility or declining collateral values, the system remains stable, backed, and functional—bolstering long-term confidence in POINT as a sustainable liquidity asset.

5.4. Phase IV: BSPT Lending

The final envisaged phase adds a lending and borrowing layer on top of tokenized real estate. BSPT holders can post their tokens as collateral to borrow POINT, opening the door to advanced DeFi strategies like leveraged property exposure or automated rental-yield repayment. Liquidations, in the event of undercollateralization, convert BSPTs into POINT, ensuring the protocol remains overcollateralized. For BST holders, an active lending market further elevates the importance of governance, as interest rates, liquidation rules, and portfolio thresholds become critical levers for protocol success.

Phase IV will introduce borrowing and lending against BSPTs, opening up leverage and more complex DeFi strategies. Borrowers can deposit BSPTs as collateral to borrow POINT. If the collateral value drops, liquidations will convert BSPTs into POINT to repay the debt position. BST governance will determine interest rates, liquidation thresholds, and lending rules, further establishing BST's essential role in the ecosystem.

Users are able to collateralize their borrow positions with multiple BSPTs. This strategy further strengthens Oceanpoint as a complete liquidity engine, changing real-estate tokens into collateral that can be traded and leveraged for DeFi loan markets.

6. System Architecture & Technical Design

Oceanpoint relies on a core set of smart contracts operating in tandem to ensure secure, transparent, and upgradeable functionality. At the heart lies the OceanpointToken (POINT), an ERC-20 token that can only be minted or burned by authorized roles within the protocol. The companion OceanpointProtocol contract manages all deposit and withdrawal logic and is the only authorized address that can mint or burn POINT. Initially, the smart contract system will process only simple stablecoin ↔ POINT transactions, but is designed to expand as new features—such as BSPT deposits and advanced discount/premium calculations—are introduced.

The system architecture also uses UUPS or proxy patterns so that upgrades can be made without forcing users to migrate to new contract addresses. These upgradability features speak to our iterative development process that will allow the protocol to move from Phase I to Phase IV and beyond with minimal disruption for users and liquidity providers. Administrative rights will initially be assigned to the Governance Board's multi-signature wallet to allow for quick responses to urgent issues. As the protocol grows in value and attracts more attention from the community, a more decentralized governance structure tied to sBST will be introduced.

Security is a top priority. To ensure system reliability and user trust, critical functions will require special admin permissions assigned to the Governance Board. Regular third-party audits and internal testing will ensure that the smart contract system can handle the complexities of bridging real estate with DeFi while maintaining user confidence.

6.1. Core Contracts & Upgradeability

6.1.1. OceanpointToken (POINT)

- **ERC-20** token, symbol: **POINT**.
- **Mint/Burn Control**: Only designated **minterBurner** addresses (e.g., the OceanpointProtocol contract) can mint/burn.
- **Ownable**: The contract owner (initially a multi-sig) can set minter/burner privileges.

JavaScript

```
/*
 * SPDX-License-Identifier: UNLICENSED
 * Copyright © 20252 Blocksquare d.o.o.
 */

pragma solidity ^0.8.28;

import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";

/// @title Oceanpoint Token

/// @notice ERC20 token that powers the Point Protocol, enabling minting and burning by
authorized addresses.
contract OceanpointToken is ERC20, Ownable {
    /// @dev Tracks which addresses are authorized to mint or burn tokens.
    mapping(address => bool) private _minterBurner;

    /// @notice Emitted when the minter/burner status of an address is updated.
    /// @param minterBurner The address whose permissions changed.
    /// @param canUse A boolean indicating whether the address is now authorized (`true`)
or not (`false`).
    event MinterBurnerStatusChanged(address minterBurner, bool canUse);

    /// @notice Error thrown when an address that is not authorized attempts to mint or
burn tokens.
    /// @param minterBurner The address that attempted the unauthorized action.
    error NoPermissionToMintBurn(address minterBurner);

    /// @dev Initializes the token with a specified name ("Oceanpoint Token") and symbol
("POINT").
```

```

/// The contract's deployer is set as the initial owner.
constructor() ERC20("Oceanpoint Token", "POINT") Ownable(_msgSender()) {}

/// @notice Mints a specified amount of tokens to a given address.
/// @dev Only callable by an address with mint/burn permissions.
/// @param account The address that will receive the newly minted tokens.
/// @param amount The number of tokens to be minted.
function mint(address account, uint256 amount) external {
    if (!_minterBurner[_msgSender()]) {
        revert NoPermissionToMintBurn(_msgSender());
    }
    _mint(account, amount);
}

/// @notice Burns a specified amount of tokens from a given address.
/// @dev Only callable by an address with mint/burn permissions.
/// @param account The address whose tokens will be burned.
/// @param amount The number of tokens to be burned.
function burn(address account, uint256 amount) external {
    if (!_minterBurner[_msgSender()]) {
        revert NoPermissionToMintBurn(_msgSender());
    }
    _burn(account, amount);
}

/// @notice Updates the mint/burn authorization status for a specific address.
/// @dev Only callable by the contract owner.
/// @param minterBurner The address whose authorization status is being changed.
/// @param canUse The new authorization status (true = authorized, false = not
authorized).
function changeMinterOrBurner(address minterBurner, bool canUse)
    external
    onlyOwner
{
    _minterBurner[minterBurner] = canUse;
    emit MinterBurnerStatusChanged(minterBurner, canUse);
}

/// @notice Checks if an address has permission to mint or burn tokens.
/// @param minterBurner The address to check.
/// @return A boolean indicating whether the address is authorized to mint/burn
(`true`) or not (`false`).
function canMintOrBurn(address minterBurner) external view returns (bool) {
    return _minterBurner[minterBurner];
}
}

```

6.1.2. OceanpointProtocol

- **UUPS Upgradeable:** Allows new logic to be attached in later phases without redeploying.
- **Deposit / Withdrawal:** Manages stablecoin ↔ POINT in Phase I, later extended for BSPTs.

- **Registered StableCoins:** The owner can register or deregister stablecoins to accept.

JavaScript

```
/*
 * SPDX-License-Identifier: UNLICENSED
 * Copyright © 2025 Blocksquare d.o.o.
 */

pragma solidity ^0.8.28;

import {OwnableUpgradeable} from
"@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol";
import {UUPSUpgradeable} from
"@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol";
import {ReentrancyGuardUpgradeable} from
"@openzeppelin/contracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

/// @notice Interface providing mint and burn functions for the Point token
interface PointProtocolHelpers {
    function mint(address to, uint256 amount) external;
    function burn(address to, uint256 amount) external;
}

/// @title Oceanpoint Protocol

/// @notice Core contract for handling deposits, withdrawals, property transactions, and
tax logic in the Oceanpoint ecosystem.
contract OceanpointProtocol is
    OwnableUpgradeable,
    ReentrancyGuardUpgradeable,
    UUPSUpgradeable
{
    using SafeERC20 for IERC20;

    // Constants used in valuation calculations
    uint256 private constant TOTAL_SUPPLY = 10 ** 5 * 1 ether;
    uint256 private constant PERCENT = 10 ** 4;

    /// @notice Holds parameters for each registered stablecoin
    /// @dev 'conversionRate' represents the token's exchange rate into POINT;
    'registered' indicates if it's active
    struct StableCoinRegistry {
        uint256 conversionRate;
        bool registered;
    }

    /// @notice Holds parameters for each registered property token
    /// @dev 'valuation' is the total value of the property, 'threshold' is the max supply
the protocol can hold, 'registered' signals if it's active
    struct PropertyRegistry {
        uint256 valuation;
        uint256 lastUpdate;
        uint256 threshold;
    }
}
```

```
        bool registered;
    }

    // Address of the Point token contract
    address private _pointToken;

    // Stable coin variables
    uint256 private _totalStableCoinValuation;
    mapping(address => StableCoinRegistry) private _registeredStableCoins;
    address[] private _stableCoins;

    // Property token variables
    uint256 private _totalPropertyTokenValuation;
    mapping(address => PropertyRegistry) private _registeredProperties;

    // Taxation parameters
    uint256 private _targetTreasuryExposure;

    uint256 private _minNAVDiscout;
    uint256 private _maxNAVDiscout;

    uint256 private _minNAVPremiumRate;
    uint256 private _maxNAVPremiumRate;

    uint256 private _minDepositTax;
    uint256 private _maxDepositTax;

    uint256 private _minWithdrawalTax;
    uint256 private _maxWithdrawalTax;

    /// @notice Emitted when a stablecoin is successfully registered
    /// @param stableCoinAddress The stablecoin's contract address
    /// @param conversionRate The stablecoin's conversion rate to POINT
    event StableCoinRegistered(address stableCoinAddress, uint256 conversionRate);

    /// @notice Emitted when a stablecoin is deactivated and can no longer be used
    /// @param stableCoinAddress The stablecoin's contract address
    event StableCoinDeregistered(address stableCoinAddress);

    /// @notice Emitted when a user deposits tokens and receives POINT
    /// @param token Address of the deposited token
    /// @param amountToken Number of tokens deposited
    /// @param amountPoint Amount of newly minted POINT tokens
    event Deposit(address token, uint256 amountToken, uint256 amountPoint);

    /// @notice Emitted when a user withdraws tokens, burning their POINT
    /// @param token Address of the withdrawn token
    /// @param amountToken Number of tokens returned
    /// @param amountPoint Amount of POINT tokens burned
    event Withdrawal(address token, uint256 amountToken, uint256 amountPoint);

    /// @notice Emitted when a property token is registered
    /// @param property Address of the property token
    /// @param threshold Maximum number of property tokens the protocol can hold
    /// @param valuation The property's assigned valuation
    event PropertyRegistered(address property, uint256 threshold, uint256 valuation);
```

```

/// @notice Emitted when a property token is deregistered
/// @param property Address of the property token
event PropertyDeregistered(address property);

/// @notice Emitted when the protocol's target treasury exposure is updated
/// @param newExposure The updated treasury exposure target
event TargetTreasuryExposureUpdated(uint256 newExposure);

/// @notice Emitted when NAV discount parameters are updated
/// @param minNavDiscount The new minimum discount
/// @param maxNavDiscount The new maximum discount
event NavDiscountUpdated(uint256 minNavDiscount, uint256 maxNavDiscount);

/// @notice Emitted when NAV premium parameters are updated
/// @param minNavPremiumRate The new minimum premium rate
/// @param maxNavPremiumRate The new maximum premium rate
event NavPremiumRateUpdated(uint256 minNavPremiumRate, uint256 maxNavPremiumRate);

/// @notice Emitted when deposit tax parameters are updated
/// @param minDepositTax The new minimum deposit tax
/// @param maxDepositTax The new maximum deposit tax
event DepositTaxUpdated(uint256 minDepositTax, uint256 maxDepositTax);

/// @notice Emitted when withdrawal tax parameters are updated
/// @param minWithdrawalTax The new minimum withdrawal tax
/// @param maxWithdrawalTax The new maximum withdrawal tax
event WithdrawalTaxUpdated(uint256 minWithdrawalTax, uint256 maxWithdrawalTax);

/// @dev Calculates the dynamic scaling factor used in premium/discount/tax
calculations
/// @return A scale factor based on the difference between the current and target
treasury exposures
function _get_scale() private view returns (uint256) {
    uint256 current = calculateCurrentTreasuryExposure();
    // If target is zero or below current, scaling factor is zero
    if (_targetTreasuryExposure == 0 || _targetTreasuryExposure < current) {
        return 0;
    }
    return (_targetTreasuryExposure - current) / _targetTreasuryExposure;
}

/// @dev Restricts the upgrade functionality to the contract owner
/// @param newImplementation Address of the new implementation
function _authorizeUpgrade(
    address newImplementation
) internal override onlyOwner {}

/// @notice Initializes the contract with the given Point token address and sets the
owner
/// @dev This can only be run once due to the `initializer` modifier
/// @param pointToken Address of the Point token contract
/// @param ownerWallet Address of the designated contract owner
function initialize(
    address pointToken,
    address ownerWallet

```

```

) external initializer {
    require(
        pointToken != address(0) && ownerWallet != address(0),
        "LPStaking: Address is zero"
    );
    __ReentrancyGuard_init();
    __UUPSUpgradeable_init();
    _pointToken = pointToken;
    _transferOwnership(ownerWallet);
}

/*****
 * STABLE COINS *
*****/

/// @notice Deposits a registered stablecoin and mints POINT to the sender
/// @dev Requires the stablecoin to be registered
/// @param token The stablecoin's address
/// @param amount The amount of stablecoin deposited
/// @custom:event Emits a `Deposit` event
function depositStableCoin(address token, uint256 amount) external nonReentrant {
    require(_registeredStableCoins[token].registered, "Stable Coin not registered!");

    // Transfer stablecoins in from the caller
    IERC20(token).transferFrom(_msgSender(), address(this), amount);

    // Calculate how many POINT tokens are minted
    uint256 points = amount * _registeredStableCoins[token].conversionRate / PERCENT;
    _totalStableCoinValuation += points;

    // Adjust minted POINT by applying the deposit tax
    uint256 amountToMint = points - (points * calculateDepositTax() / PERCENT);
    PointProtocolHelpers(_pointToken).mint(_msgSender(), amountToMint);

    emit Deposit(token, amount, amountToMint);
}

/// @notice Burns POINT and withdraws the corresponding amount of a registered
stablecoin
/// @dev Requires sufficient balance of the stablecoin in this contract
/// @param token The stablecoin's address
/// @param amount The amount of POINT tokens to burn
/// @custom:event Emits a `Withdrawal` event
function withdrawStableCoin(address token, uint256 amount) external nonReentrant {
    require(
        IERC20(token).balanceOf(address(this)) >= amount,
        "Insufficient Stable Coin balance!"
    );

    // Calculate how many stablecoins are returned by burning 'amount' of POINT
    uint256 coin = amount * PERCENT / _registeredStableCoins[token].conversionRate;

    // Adjust returned stablecoin amount by applying the withdrawal tax
    uint256 amountToReturn = coin - (coin * calculateWithdrawalTax() / PERCENT);
    _totalStableCoinValuation -= amountToReturn;
}

```

```

        // Transfer the adjusted amount back to the caller
        IERC20(token).transfer(_msgSender(), amountToReturn);

        // Burn the caller's POINT tokens
        PointProtocolHelpers(_pointToken).burn(_msgSender(), amount);

        emit Withdrawal(token, amountToReturn, amount);
    }

    /// @notice Registers a new stablecoin or updates the conversion rate of an existing
    one
    /// @dev Conversion rate format: 12000 is 1.2, 10000 is 1
    /// @param stableCoin The stablecoin's address
    /// @param conversionRate Conversion rate to POINT
    /// @custom:event Emits `StableCoinRegistered`
    function registerStableCoin(address stableCoin, uint256 conversionRate) external
    onlyOwner {
        require(conversionRate > 0, "Conversion rate must be > 0");
        if (_registeredStableCoins[stableCoin].conversionRate == 0) {
            _stableCoins.push(stableCoin);
        }
        _registeredStableCoins[stableCoin].registered = true;
        _registeredStableCoins[stableCoin].conversionRate = conversionRate;

        emit StableCoinRegistered(stableCoin, conversionRate);
    }

    /// @notice Deregisters an existing stablecoin
    /// @param stableCoin The stablecoin's address
    /// @custom:event Emits `StableCoinDeregistered`
    function deregisterStableCoin(address stableCoin) external onlyOwner {
        require(_registeredStableCoins[stableCoin].registered, "Stable Coin needs to
    registered first!");
        _registeredStableCoins[stableCoin].registered = false;

        emit StableCoinDeregistered(stableCoin);
    }

    /// @notice Recalculates the total stablecoin valuation in the contract
    /// @dev Combines balances of all registered stablecoins, factoring in their
    conversion rates
    function consolidateTotalStableCoinValuation() external onlyOwner {
        uint256 currentValuation = 0;
        for (uint256 i = 0; i < _stableCoins.length; i++) {
            address stableCoin = _stableCoins[i];
            currentValuation += IERC20(stableCoin).balanceOf(address(this))
                * _registeredStableCoins[stableCoin].conversionRate
                / PERCENT;
        }
        _totalStableCoinValuation = currentValuation;
    }

    /*****
    /* PROPERTY TOKENS */
    *****/

```

```

    /// @notice Sells property tokens to this contract in return for newly minted POINT
    /// @dev Requires the property token to be registered and under the threshold limit
    /// @param property Property token's address
    /// @param amount Amount of property tokens to sell
    /// @custom:event Emits `Deposit`
    function sellPropertyTokenToOceanpoint(address property, uint256 amount) external
    nonReentrant {
        require(_registeredProperties[property].registered, "Property token must be
    registered!");
        require(
            IERC20(property).balanceOf(address(this)) + amount <=
    _registeredProperties[property].threshold,
            "Threshold exceeded!");
    };

    // Transfer property tokens from caller to the protocol
    IERC20(property).transferFrom(_msgSender(), address(this), amount);

    // Update the protocol's internal valuation
    uint256 propTokenValuation = amount
        * _registeredProperties[property].valuation
        / TOTAL_SUPPLY;
    _totalPropertyTokenValuation += propTokenValuation;

    // Mint POINT to the seller, applying NAV discount
    uint256 amountToMint = propTokenValuation - (propTokenValuation *
    calculateNAVDiscount() / PERCENT);
    PointProtocolHelpers(_pointToken).mint(_msgSender(), amountToMint);

    emit Deposit(property, amount, amountToMint);
}

    /// @notice Buys property tokens from the protocol by burning the caller's POINT
    /// @dev Requires sufficient property tokens in the protocol and the property must be
    registered
    /// @param property Property token's address
    /// @param amount Amount of POINT to burn
    /// @custom:event Emits `Withdrawal`
    function buyPropertyTokenFromOceanpoint(address property, uint256 amount) external
    nonReentrant {
        require(IERC20(property).balanceOf(address(this)) >= amount, "Insufficient
    property token balance!");

        // Burn POINT from the caller
        PointProtocolHelpers(_pointToken).burn(_msgSender(), amount);

        // Determine how many property tokens are returned
        uint256 propTokenAmount = amount * TOTAL_SUPPLY /
    _registeredProperties[property].valuation;

        // Adjust returned tokens by applying NAV premium
        uint256 amountToReturn = propTokenAmount - (propTokenAmount *
    calculateNAVPremium() / PERCENT);
        IERC20(property).transfer(_msgSender(), amountToReturn);

        // Update internal property valuation

```

```

        _totalPropertyTokenValuation -= (
            amountToReturn * _registeredProperties[property].valuation
            / TOTAL_SUPPLY
        );

        emit Withdrawal(property, amountToReturn, amount);
    }

    /// @notice Registers a new property token or updates an existing one
    /// @param property Property token's address
    /// @param threshold Maximum token amount the protocol can hold
    /// @param valuation The property token's total valuation
    /// @custom:event Emits `PropertyRegistered`
    function registerProperty(address property, uint256 threshold, uint256 valuation)
external onlyOwner {
        _registeredProperties[property].registered = true;
        _registeredProperties[property].threshold = threshold;
        _registeredProperties[property].valuation = valuation;
        _registeredProperties[property].lastUpdate = block.timestamp;

        emit PropertyRegistered(property, threshold, valuation);
    }

    /// @notice Deactivates an existing property token
    /// @param property Property token's address
    /// @custom:event Emits `PropertyDeregistered`
    function deregisterProperty(address property) external onlyOwner {
        require(!_registeredProperties[property].registered, "Property token is not
registered!");
        _registeredProperties[property].registered = false;

        emit PropertyDeregistered(property);
    }

    /*****
    * TAXATION *
    *****/

    /// @notice Sets the protocol's target treasury exposure
    /// @dev Used when calculating premiums, discounts, and taxes
    /// @param exposure The new exposure target
    /// @custom:event Emits `TargetTreasuryExposureUpdated`
    function setTargetTreasuryExposure(uint256 exposure) external onlyOwner {
        _targetTreasuryExposure = exposure;
        emit TargetTreasuryExposureUpdated(exposure);
    }

    /// @notice Updates the minimum and maximum NAV discount
    /// @dev Used when selling property tokens to the protocol
    /// @param minNAVDiscount New minimum discount
    /// @param maxNAVDiscount New maximum discount
    /// @custom:event Emits `NavDiscountUpdated`
    function setNavDiscount(uint256 minNAVDiscount, uint256 maxNAVDiscount) external
onlyOwner {
        require(maxNAVDiscount >= minNAVDiscount, "Max must be greater than Min");
        _minNAVDiscount = minNAVDiscount;
    }

```

```

        _maxNAVDiscount = maxNAVDiscount;

        emit NavDiscountUpdated(minNAVDiscount, maxNAVDiscount);
    }

    /// @notice Updates the minimum and maximum NAV premium rate
    /// @dev Used when buying property tokens from the protocol
    /// @param minNAVPremiumRate New minimum premium
    /// @param maxNAVPremiumRate New maximum premium
    /// @custom:event Emits `NavPremiumRateUpdated`
    function setNavPremiumRate(uint256 minNAVPremiumRate, uint256 maxNAVPremiumRate)
    external onlyOwner {
        require(maxNAVPremiumRate >= minNAVPremiumRate, "Max must be greater than Min");
        _minNAVPremiumRate = minNAVPremiumRate;
        _maxNAVPremiumRate = maxNAVPremiumRate;

        emit NavPremiumRateUpdated(minNAVPremiumRate, maxNAVPremiumRate);
    }

    /// @notice Updates the minimum and maximum deposit tax
    /// @dev Used when depositing stablecoins into the protocol
    /// @param minDepositTax New minimum deposit tax
    /// @param maxDepositTax New maximum deposit tax
    /// @custom:event Emits `DepositTaxUpdated`
    function setDepositTax(uint256 minDepositTax, uint256 maxDepositTax) external
    onlyOwner {
        require(maxDepositTax >= minDepositTax, "Max must be greater than Min");
        _minDepositTax = minDepositTax;
        _maxDepositTax = maxDepositTax;

        emit DepositTaxUpdated(minDepositTax, maxDepositTax);
    }

    /// @notice Updates the minimum and maximum withdrawal tax
    /// @dev Used when withdrawing stablecoins from the protocol
    /// @param minWithdrawalTax New minimum withdrawal tax
    /// @param maxWithdrawalTax New maximum withdrawal tax
    /// @custom:event Emits `WithdrawalTaxUpdated`
    function setWithdrawalTax(uint256 minWithdrawalTax, uint256 maxWithdrawalTax) external
    onlyOwner {
        require(maxWithdrawalTax >= minWithdrawalTax, "Max must be greater than Min");
        _minWithdrawalTax = minWithdrawalTax;
        _maxWithdrawalTax = maxWithdrawalTax;

        emit WithdrawalTaxUpdated(minWithdrawalTax, maxWithdrawalTax);
    }

    // TODO Potential revenue-related logic can be added here

    /**
     * GETTERS */

    /// @notice Returns the protocol's total stablecoin valuation
    /// @return Total amount of stablecoins in POINT-equivalent terms
    function getTotalStableCoinValuation() public view returns (uint256) {

```

```

        return _totalStableCoinValuation;
    }

    /// @notice Checks if a given stablecoin is registered
    /// @param stableCoin The stablecoin's address
    /// @return True if registered, false otherwise
    function isRegisteredStableCoin(address stableCoin) external view returns (bool) {
        return _registeredStableCoins[stableCoin].registered;
    }

    /// @notice Retrieves the registration details of a property token
    /// @param property Property token's address
    /// @return A `PropertyRegistry` struct with details about the property token
    function getRegisteredPropertyTokenInfo(address property) external view returns
(PropertyRegistry memory) {
        return _registeredProperties[property];
    }

    /// @notice Calculates the protocol's current treasury exposure as a ratio of property
tokens to stablecoin valuation
    /// @return Current exposure ratio
    function calculateCurrentTreasuryExposure() public view returns (uint256) {
        if(_totalStableCoinValuation == 0) {
            return 0;
        }
        return _totalPropertyTokenValuation / _totalStableCoinValuation;
    }

    /// @notice Calculates the dynamic NAV discount based on the protocol's treasury
exposure
    /// @dev Used when selling property tokens to the protocol
    /// @return NAV discount percentage (scaled by `PERCENT`)
    function calculateNAVDiscount() public view returns (uint256) {
        return _maxNAVDiscount - (_maxNAVDiscount - _minNAVDiscount) * _get_scale();
    }

    /// @notice Calculates the dynamic NAV premium rate based on the protocol's treasury
exposure
    /// @dev Used when buying property tokens from the protocol
    /// @return NAV premium percentage (scaled by `PERCENT`)
    function calculateNAVPremium() public view returns (uint256) {
        return _minNAVPremiumRate + (_maxNAVPremiumRate - _minNAVPremiumRate) *
_get_scale();
    }

    /// @notice Calculates the dynamic deposit tax based on the protocol's treasury
exposure
    /// @dev Used when depositing stablecoins
    /// @return Deposit tax percentage (scaled by `PERCENT`)
    function calculateDepositTax() public view returns (uint256) {
        return _minDepositTax + (_maxDepositTax - _minDepositTax) * _get_scale();
    }

    /// @notice Calculates the dynamic withdrawal tax based on the protocol's treasury
exposure
    /// @dev Used when withdrawing stablecoins

```

```
/// @return Withdrawal tax percentage (scaled by `PERCENT`)
function calculateWithdrawalTax() public view returns (uint256) {
    return _maxWithdrawalTax - (_maxWithdrawalTax - _minWithdrawalTax) * _get_scale();
}

/// @notice Returns the protocol's total property token valuation
/// @return Total property valuation in POINT-equivalent terms
function getTotalPropertyTokenValuation() external view returns (uint256) {
    return _totalPropertyTokenValuation;
}
}
```

6.1.3. OceanpointRouter (OPR)

The OceanpointRouter (OPR) serves as the transactional hub for tokenized real estate within the Oceanpoint ecosystem, simplifying the movement of assets while ensuring optimal execution for users. Rather than requiring multiple transactions, OPR aggregates and automates complex swaps, allowing users to seamlessly move between BSPTs, POINT, and stablecoins in a single interaction.

At its core, OPR acts as a fee-optimized routing mechanism, efficiently bridging the Blocksquare Property Token (BSPT) system with Oceanpoint's liquidity engine. Given that BSPT transfers typically incur a transfer tax, OPR is designated as a fee-exempt address, ensuring BSPTs can be acquired, traded, and integrated into the marketplace without unnecessary costs. This allows for frictionless participation in the ecosystem while unlocking deeper liquidity for tokenized real estate.

Beyond simplifying transactions, OPR dynamically sources liquidity from multiple venues, ensuring the most efficient execution path. Depending on market conditions, BSPT transactions may be routed through the POINT system, 0x limit orders, or automated market makers (AMMs), optimizing cost and minimizing slippage. Similarly, users seeking to acquire POINT tokens may do so via direct minting or secondary market purchases, with OPR determining the most favorable option.

Routing & Execution

OPR dynamically evaluates liquidity sources to ensure each transaction is processed at the most favorable rate while maintaining execution efficiency.

Acquiring BSPTs	Selling BSPTs
Users seeking to acquire BSPTs initiate a transaction via OPR, which evaluates multiple options: • Oceanpoint's POINT system: Direct BSPT minting via POINT. • 0x Protocol: Fulfills limit orders from users selling BSPTs. • AMM Liquidity Pools: Executes swaps when market conditions are favorable. • Hybrid Execution: Splits the order between different sources to optimize price. <i>If, for example, an existing 0x sell order is priced below the POINT system's acquisition rate, OPR will prioritize fulfilling that order before turning to other sources.</i>	For users liquidating BSPTs, OPR optimizes execution by evaluating: • Oceanpoint's POINT system: Selling BSPTs for POINT, redeemable for stablecoins. • 0x Protocol limit orders: Matching with existing buyers at optimal prices. • AMM Liquidity Pools: Providing an additional exit route when necessary. • Hybrid Execution: Splits the order between different sources to optimize price. <i>By dynamically routing trades based on market conditions, OPR ensures that users receive the best possible execution while maintaining ecosystem stability.</i>

Acquiring POINT Tokens	Selling POINT Tokens
For users looking to acquire POINT there are two primary options available: • Direct Minting: Depositing stablecoins to receive new POINT tokens. • Secondary Market Purchase: Acquiring POINT from Uniswap or other liquidity pools when more cost-effective than minting. <i>OPR automatically selects the lowest-cost route to maximize efficiency, ensuring that users always get POINT at the best available rate.</i>	For users looking to sell POINT there are two primary options available: • Direct Burning: Burning POINT tokens to receive stablecoins from the protocol. • Secondary Market Purchase: Selling POINT to Uniswap or other liquidity pools when more cost-effective than burning. <i>OPR automatically selects the lowest-cost route to maximize efficiency, ensuring that users always sell POINT at the best available rate.</i>

Off-Chain Routing Algorithm & UI Flow

Due to the complexity of transactions and the need for optimal routing, OPR incorporates an off-chain routing algorithm. This algorithm determines the best execution path by evaluating various factors such as liquidity, transaction costs, and market conditions. Once the optimal route is determined, this information is presented to the user in the UI, allowing them to confirm or adjust the transaction before execution.

The routing decision-making process is entirely off-chain, meaning that the system calculates the best route and then presents it to the user for final confirmation, ensuring that transactions are efficient and cost-effective. This approach minimizes the complexity of on-chain operations while providing transparency and control to the user during the transaction process.

Security & Price Optimization

To ensure robust execution while safeguarding users from market inefficiencies, OPR integrates multiple security and optimization mechanisms.

Slippage Protection

Users can set slippage tolerances to prevent unintended losses from price fluctuations. If a transaction exceeds the defined threshold, it is automatically reverted.

Oracle-Based Price Validation

To mitigate price manipulation risks, OPR fetches real-time BSPT and POINT price data from multiple sources, including:

- **Chainlink oracles**
- **Uniswap TWAP pricing**
- **Blocksquare's proprietary data feeds**

Before executing a transaction, OPR cross-references these sources, ensuring fair execution prices. If a price deviation exceeds a set threshold, the trade is paused to prevent manipulation.

MEV & Front-Running Protection

- **Private transactions** via Flashbots or similar relayers to prevent front-running.
- **Randomized execution orders** to mitigate sandwich attacks.

Smart Liquidity Routing

- **Orders** are dynamically split between multiple sources to reduce price impact.

- **Routing** is optimized in real-time, ensuring the most capital-efficient trade path.

Roadmap & Future Enhancements

The OceanpointRouter is built with upgradeability in mind, allowing for continuous improvements as new liquidity sources and risk management strategies emerge.

Future updates will focus on:

- **Expanded liquidity integrations** (additional DeFi pools beyond Uniswap and 0x).
- **Automated rebalancing** to optimize liquidity distribution across stablecoins, POINT, and BSPTs.
- **Dynamic batching mechanisms** to further reduce transaction costs.
- **Predictive trade execution** using historical price models.

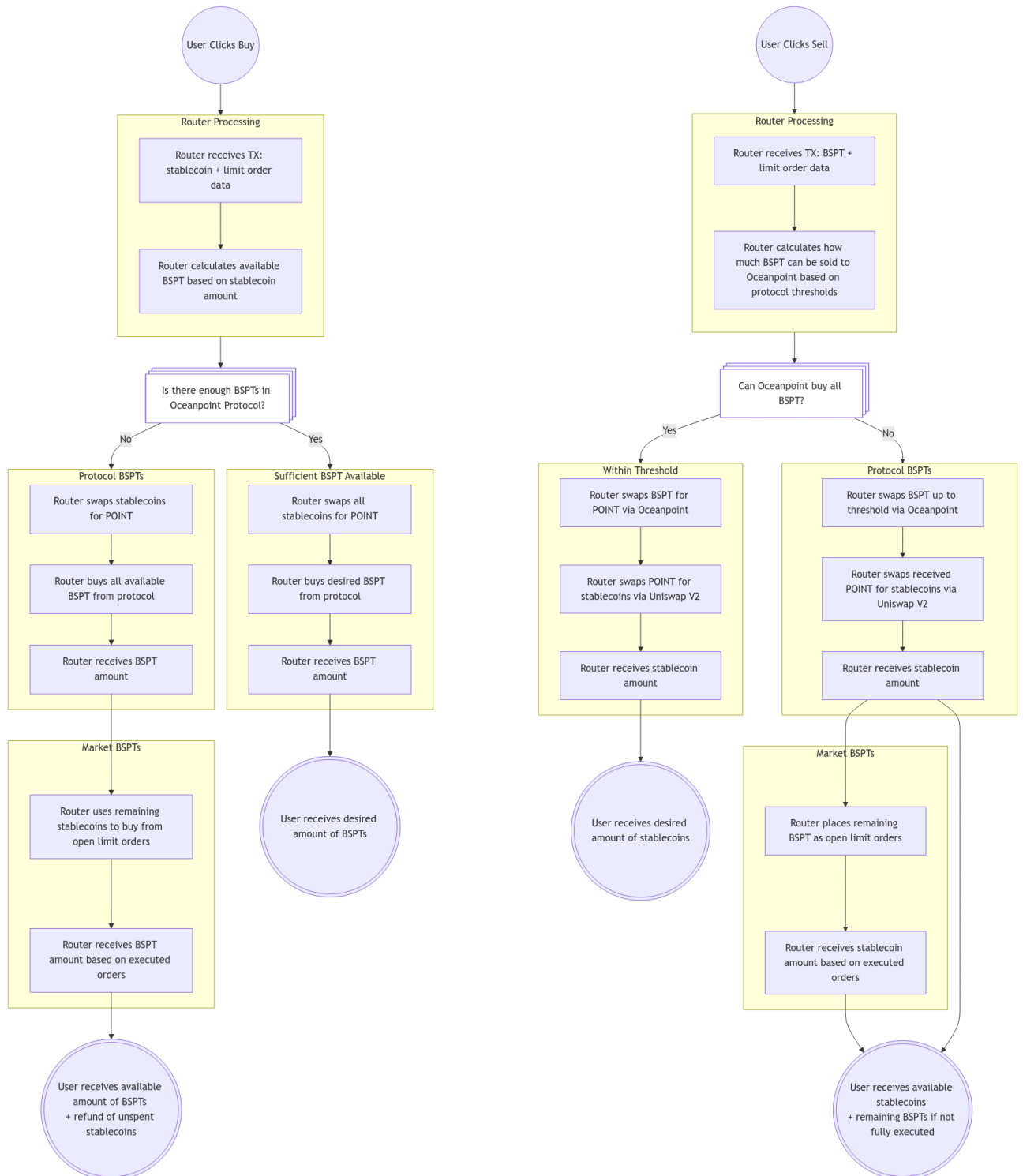
By continually refining OPR's capabilities, Oceanpoint will enhance capital efficiency, reduce user friction, and further bridge the gap between DeFi and tokenized real estate.

Summary

The OceanpointRouter (OPR) is a critical infrastructure layer for unlocking seamless liquidity within Oceanpoint. By optimizing trade execution, minimizing fees, and integrating security mechanisms, OPR ensures that users can move between BSPTs, POINT, and stablecoins with minimal friction.

Its dynamic routing, fee-exempt structure, and MEV-resistant design make it an essential component in scaling tokenized real estate within DeFi. As Oceanpoint evolves, continuous upgrades to OPR will drive greater efficiency, deeper liquidity, and a more robust tokenized asset marketplace.

6.2. Buying/Selling BSPTs Flowcharts



6.3. Example Flows (Oceanpoint)

6.3.1. Deposit Stablecoins → Mint POINT

1. User Approval

The *user* approves the Oceanpoint Protocol contract to spend a specified amount of their stablecoins (e.g., USDC) by calling the `approve` function on the stablecoin's contract:

```
stablecoin.approve(address(OceanpointProtocol), amount);
```

2. Deposit Stablecoins

The *user* deposits the approved stablecoins into the Oceanpoint Protocol by calling the `depositStableCoin` function:

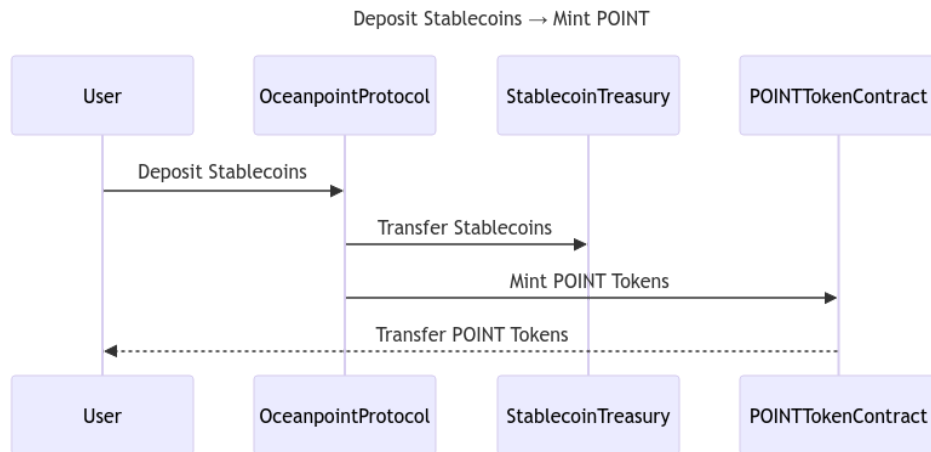
```
OceanpointProtocol.depositStableCoin(stablecoinAddress, amount);
```

3. Calculate Amount to Receive

The *contract* uses the fixed 1:1 exchange rate, and applies the `StablecoinDepositTax` based on the Dynamic Rate Function.

4. Mint POINT Tokens

The protocol mints the corresponding `amountPOINT` to the user's `address`.



6.3.2. Burn POINT → Withdraw Stablecoins

1. Burn POINT Tokens

The *user* initiates the withdrawal by calling the `withdrawStableCoin` function, specifying

the desired stablecoin:

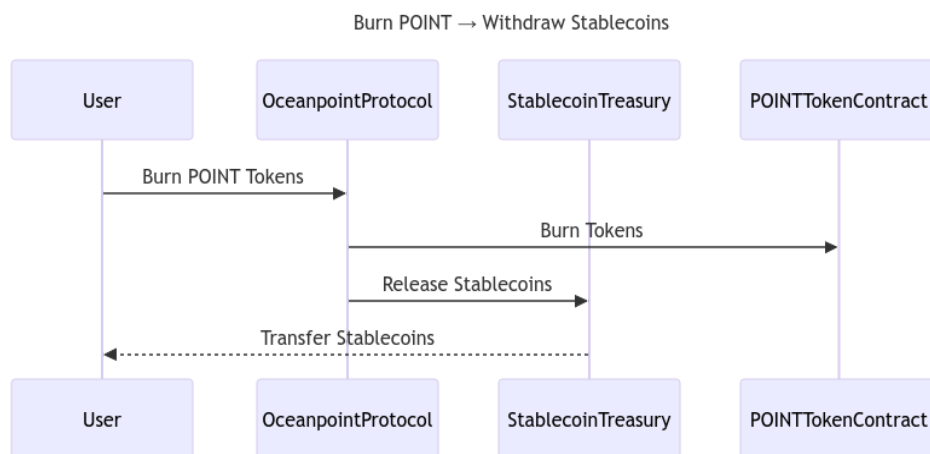
```
OceanpointProtocol.withdrawStableCoin(stablecoinAddress, amount);
```

2. Calculate Amount to Receive

The *contract* uses the fixed 1:1 exchange rate, and applies the `StablecoinWithdrawalTax` based on the Dynamic Rate Function.

3. Transfer Stablecoins

The *protocol* transfers the corresponding `amountStablecoin` to the user's `address`.



6.3.3. Sell BSPTs → Receive POINT (Mint)

1. User Approval

The *user* approves the Oceanpoint Protocol contract to spend a specified amount of their BSPT tokens: `BSPT.approve(address(OceanpointProtocol), amount);`

2. Sell BSPTs

The *user* sells the approved BSPTs to the protocol by calling the `sellPropertyTokenToOceanpoint` function:

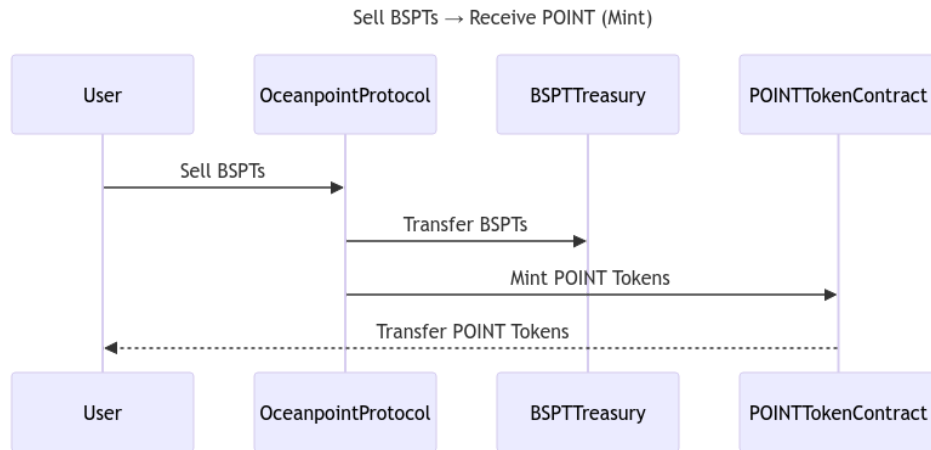
```
OceanpointProtocol.sellPropertyTokenToOceanpoint(BSPTAddress, amount);
```

3. Calculate Amount to Receive

The *contract* uses the `propertyValuation` rate, and applies the `NAVDiscountRate` based on the Dynamic Rate Function.

4. Mint POINT Tokens

The *protocol* mints the corresponding `amountPOINT` to the user's `address`.



6.3.4. Pay POINT → Buy BSPTs (Burn)

1. Buy BSPTs

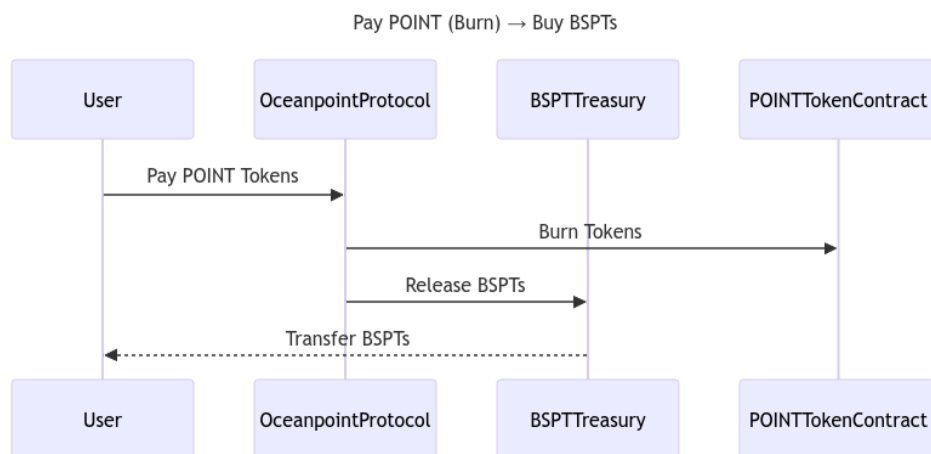
The *user* initiates the purchase by calling the `buyBSPT` function, specifying the desired BSPT: `OceanpointProtocol.buyBSPT(BSPTAddress, amount);`

2. Calculate Amount to Receive

The *contract* uses the `propertyValuation` rate, and applies the `NAVPremiumRate` based on the Dynamic Rate Function.

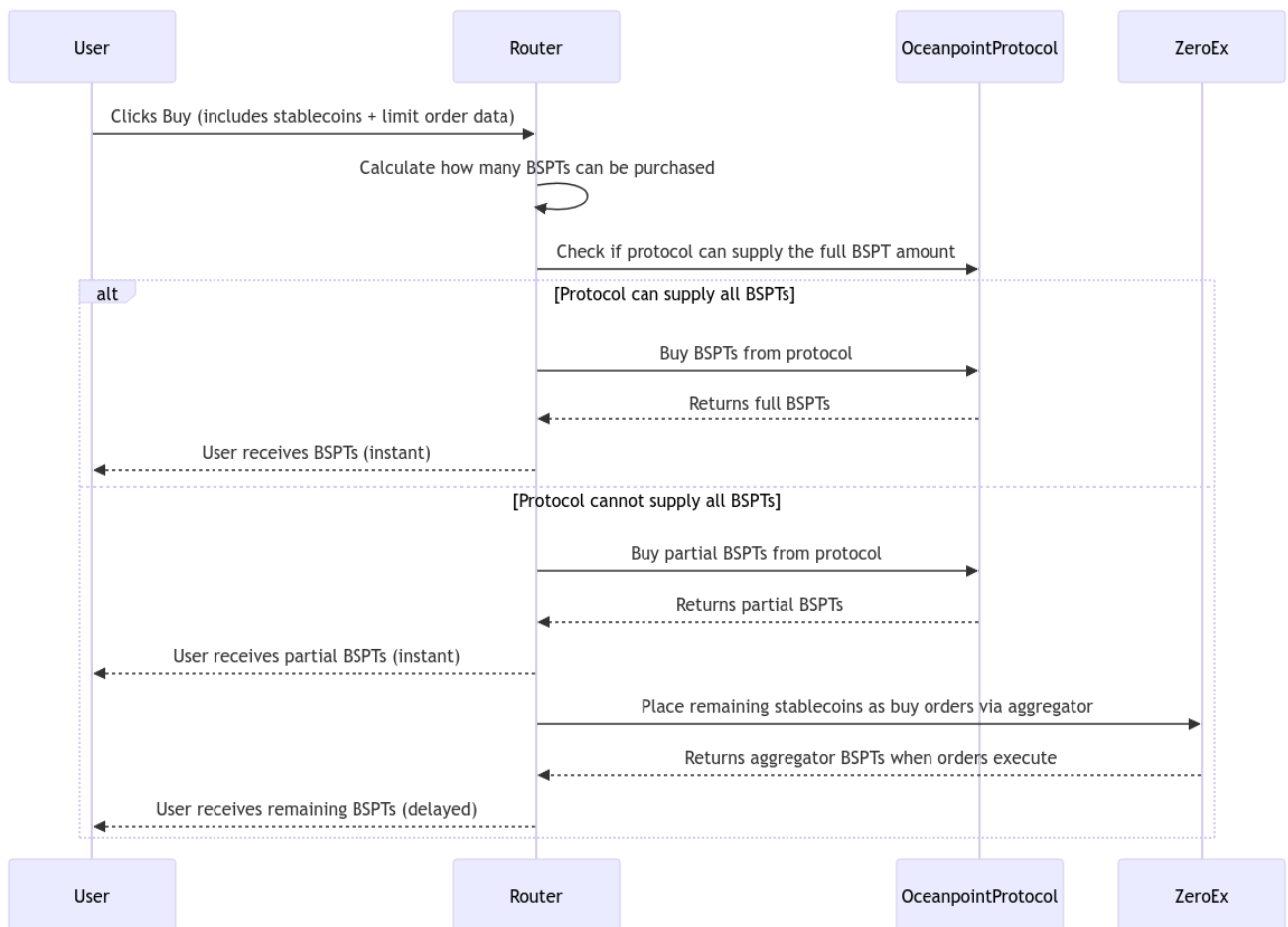
3. Transfer BSPTs

The *protocol* transfers the corresponding `amountBSPT` to the user's `address`.

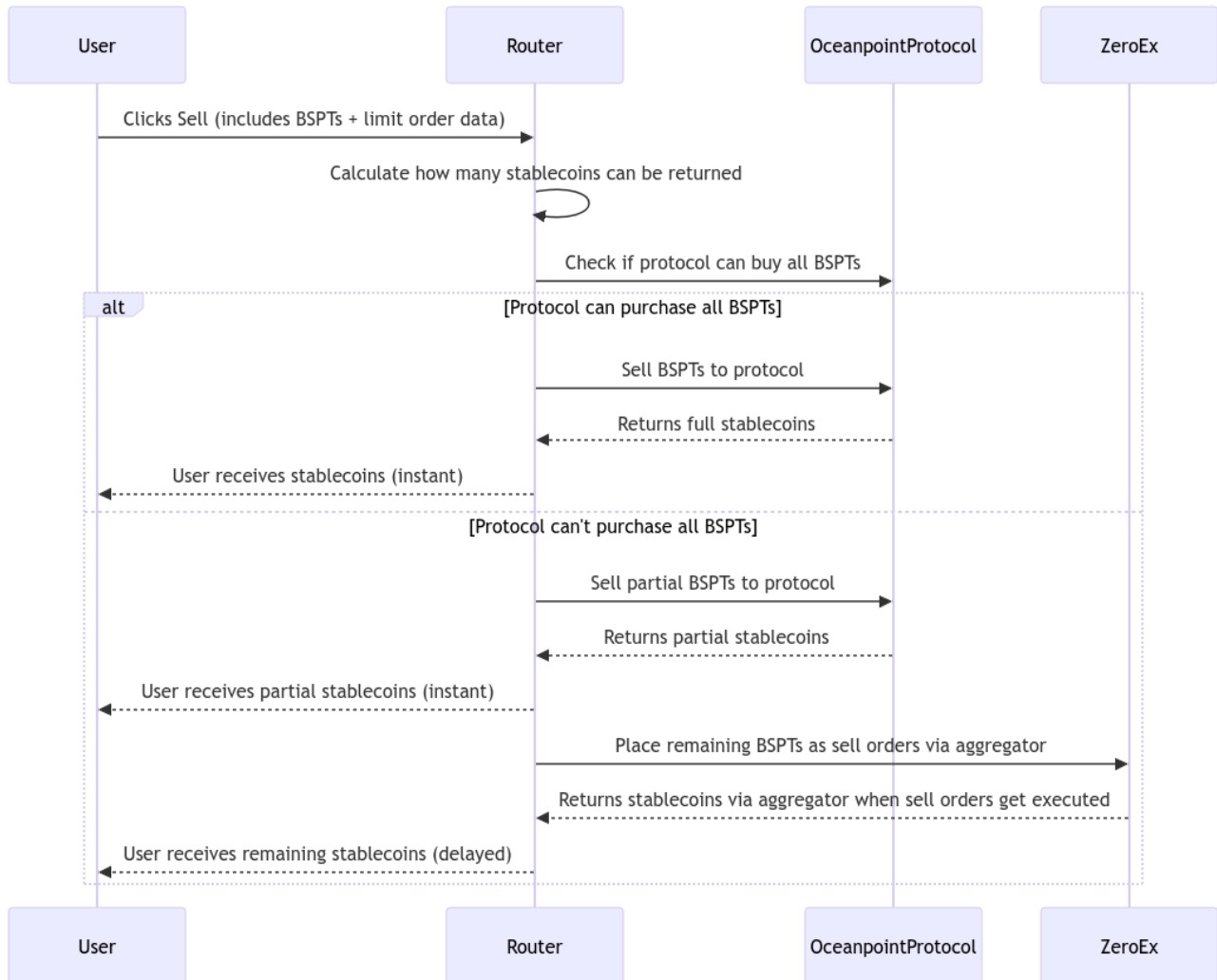


6.4. Example Flows (Marketplaces)

6.4.1. Sell Stablecoins → Get BSPTs



6.4.2. Sell BSPTs → Get Stablecoins



7. Risk Management & Governance

Oceanpoint's risk management philosophy emphasizes collateral stability and measured governance. A carefully tuned system of discounts and premiums ensures that the protocol always earns on real estate tokens NAV (Net Asset Value), driving the protocol towards overcollateralization.

In doomsday scenarios, the “POINT Zero” mechanism stands ready to protect the system if the collateral ratio threatens to slip below targeted levels. This design ensures that the interests of BST holders remain deeply interwoven with the health and growth of the overall system.

7.1. Dynamic Rate Adjustments

One of Oceanpoint’s defining capabilities is its adaptive mechanism that alters Net Asset Value (NAV) premiums and discounts, adjusting stablecoin deposit and withdrawal rates in real time. The system continuously measures the deviation $d = R - R_{target}$, where R represents the treasury’s current BSPT ratio and R_{target} denotes the protocol’s ideal composition.

Whenever $d \geq 0$, the system recognizes that it has reached or exceeded its desired BSPT exposure. In this scenario, it locks in a high NAV Discount Rate, making new BSPT deposits less appealing, and sets a minimal NAV Premium Rate, offering less incentive to buy BSPTs from the treasury. Conversely, if the treasury is underweight $d < 0$, the protocol calculates a scale factor that grows with the magnitude of $|d|$. This factor can be linear in simpler implementations or take an exponential form to amplify the response when the system is far from the target ratio. As the scale increases, the NAV Discount Rate smoothly transitions downward from its maximum level, while the NAV Premium Rate rises from a lower baseline, providing users with more favorable terms to deposit BSPTs in exchange for POINT.

The same principle applies to stablecoin deposits and withdrawals, but in an inverted manner. If the system is overweight in BSPTs, it aims to attract stablecoins by lowering deposit taxes and raising stablecoin withdrawal taxes. If the system is underweight in BSPTs, stablecoin deposits become more expensive, and withdrawals become cheaper. This feedback loop gently or aggressively nudges user behaviors so that the treasury ratio gravitates back toward R_{target} .

From a technical standpoint, each deposit or redemption transaction references the current ratio and applies a function to calculate the user’s final mint or burn amount. Governance via BST ensures that community stakeholders ultimately decide how quickly or strongly these dynamic rates should change. While the community should align in growing the protocol’s overcollateralization to protect it from extreme market downtrends as well as general crypto volatility, users i.e. real estate token holders will demand to have a convenient, reliable, stable and above all predictable route to liquidity.

In essence, the challenge lies in striking a delicate balance. On one hand, there's the necessity to provide token holders with the flexibility to liquidate their assets efficiently. On the other hand, the protocol must be fortified against potential risks that could compromise its stability and the value of the underlying assets. This balancing act will be crucial to fostering a healthy and sustainable ecosystem that benefits all participants.

7.2. Overcollateralization & In-protocol Exchange Rate

As the stablecoin or BSPT backing grows beyond the circulating supply of POINT, and should the market price constantly show quotes above the in-protocol 1:1 rate, the system could incrementally adjust the internal exchange rate in favor of POINT. Rather than a strict peg, as such, POINT adopts a “flatcoin-like” mechanism that traces the real-world growth of tokenized real estate. Over time, as the total value locked (TVL) expands, each POINT may be redeemable for slightly more stablecoins or BSPTs, reflecting the protocol’s rising collateral coverage.

Rather than a strict peg, POINT adopts a “flatcoin-like” mechanism, which closely tracks the real-world growth of tokenized real estate. This system allows for Governance to make adjustments to the internal exchange rate, potentially benefiting POINT if its market price remains above the 1:1 rate and the backing of stablecoins or BSPTs exceeds the circulating POINT supply.

As the total value locked (TVL) increases over time, the exchange rate may shift, enabling each POINT to be redeemed for a slightly greater amount of stablecoins or BSPTs. This adjustment reflects the protocol's growing collateral coverage, enhancing the stability and value of POINT tokens.

To summarize, the system aims to maintain a balance between POINT's market price and its underlying collateralization, ensuring that the token remains robustly backed while also allowing for potential appreciation in value as the ecosystem expands.

7.3. Managing Undercollateralization Scenarios

We have developed mitigation strategies to effectively manage undercollateralization. These strategies were established through various simulations and stress tests that considered scenarios such as a decline in the property market leading to a reduction in Net Asset Value (NAV), or a downturn in the cryptocurrency market that diminishes liquidity and devalues POINT tokens.

7.3.1. Mitigation 1: Increasing Spreads

The primary aim of this mitigation strategy is to enhance the protocol's collateralization by modifying financial parameters to generate higher revenues through increased spreads.

Specifically, the protocol seeks to improve its collateral position by raising taxes on stablecoin deposits and withdrawals. Additionally, by increasing NAV premiums and discounts, the protocol can widen the spread between the buying and selling prices of BSPTs, thereby earning more from these

transactions. These adjustments require active governance decisions, ensuring that changes are deliberated and implemented with the protocol's long-term stability in mind.

This governance-led approach allows the community to respond to undercollateralization scenarios by enacting measures that bolster the protocol's financial health, ensuring resilience during market fluctuations.

7.3.2. Mitigation 2: Active Management of BSPT Purchase Discounts

The protocol actively manages the spread between BSPT purchase discounts (NAV Discount Rate) and sales premiums (NAV Premium Rate) to control the circulation of POINT tokens relative to the Total Value Locked (TVL), which encompasses both BSPTs and stablecoins. This strategy involves several stages:

1. **Stage 1:** If the collateralization rate exceeds 100%, protocol's governance is advised to set the Maximum NAV Premium Rate close to zero, encouraging the acquisition of BSPTs to balance the treasury composition.
2. **Stage 2:** If the collateralization rate falls below 100%, protocol's governance is advised to set the Minimum NAV Premium Rate to correspond with the collateral deficit, incentivizing actions that restore the desired collateral levels.
3. **Stage 3:** In situations where the collateralization rate remains below 100% and the market price of POINT tokens drops below \$1, the protocol's governance is advised to cap the Maximum NAV Premium Rate to prevent excessive premiums that could further destabilize the system.
4. **Stage 4:** To achieve a state referred to as "POINT ZERO," where equilibrium between TVL and POINT's circulating supply is restored, the protocol's governance is advised to release stablecoins from the treasury to buy and burn POINT, thereby reducing the circulating supply and aiming to restore the collateralization ratio to the target level.

These mitigation strategies are designed to maintain the stability and integrity of the Oceanpoint ecosystem during periods of market volatility, ensuring that the protocol remains resilient and well-collateralized.

7.4. Governance Roadmap

The Oceanpoint protocol is currently overseen by a Governance Board comprising six members, with applications open for new members to potentially apply. This board manages the protocol through a multi-signature (multi-sig) wallet, ensuring decisions are made collectively to maintain security and integrity. This structure allows for prompt responses to urgent matters and provides flexibility during the protocol's early development stages.

As the protocol evolves, there is a planned transition toward a more decentralized governance framework. In future phases, this will enable the broader community to propose and vote on protocol upgrades, including adjustments to discount thresholds, managing the list of accepted stablecoins, approvals for new property tokens, and updating thresholds for already approved BSPTs. This approach ensures that the system can adapt effectively during its initial development while ultimately aligning with the decentralized ethos valued by many DeFi participants.

Starting from sometime after Phase III, BST token holders will play a central role in the governance process. They will have the authority to vote on key protocol parameters that determine how the protocol responds to market changes. This design ensures that BST holders' interests remain closely aligned with the overall health and growth of the system.

8. Simulations & Stress Testing

8.1.1. Simulation Design and Purpose

The Oceanpoint protocol utilizes dynamic rate adjustments as its primary resource when managing:

- a. **Treasury composition ratio:** Arriving at, and maintaining a pre-defined ratio of Stablecoins to BSPT's tokens e.g., 80% stablecoins to 20% BSPT's.
- b. **Collateralization ratio:** Ensuring that POINT remains fully backed, that is, the value of Oceanpoint (Stablecoin + BSPT) is $\geq$ the circulating supply of point tokens based on an initial value of 1:1.

Tests took place in 3 phases:

- **Test Phase 1:** Baseline Simulations (System Stability Under Normal Market Conditions)
- **Test Phase 2:** Stress Testing (Handling Market Volatility & Stress Events)
- **Test Phase 3:** Multiple-continuous stress events.

8.1.2. Test Phase 1: Baseline Simulations

Oceanpoint's dynamic rate system was tested through four simulations designed to evaluate its ability to rebalance the treasury composition under normal market conditions. Each simulation examined different starting BSPT ratios and their progression toward predefined targets. The results demonstrated that, regardless of whether the system began in an underweight (0% BSPT), overweight (80% BSPT), or intermediate state (20% BSPT), the protocol's adaptive mechanisms effectively steered the treasury toward equilibrium. By leveraging non-linear (cubic) adjustments in

NAV discounts, premiums, and stablecoin tax rates, the system ensured controlled, gradual convergence toward the desired balance. This ability to self-correct under standard market conditions reinforces confidence in the protocol's capacity to maintain stability and proper collateralization, laying a strong foundation for more complex stress testing in future phases.

Simulation No.	Initial BSPT Ratio	Target BSPT Ratio
#1	0%	20%
#2	0%	80%
#3	20%	0%
#4	80%	20%

Results & Insights

Across all four tests, the baseline simulations confirm that Oceanpoint's dynamic rate system effectively steers the treasury composition toward its target balance, ensuring system stability and proper collateralization. Regardless of whether the system starts from an underweight BSPT state (0%), an overweight state (80%), or an intermediate composition (20%), the rate system reliably adjusts the balance over time.

A key factor in this stabilization is the non-linear (cubic) rate adjustments applied to:

- NAV discounts & premiums, influencing BSPT trading incentives.
- Stablecoin tax rates, moderating inflows and outflows.

These mechanisms ensure a controlled, gradual convergence toward the target treasury ratio (e.g., 80/20 or 0/100). What's more, the dynamic rate adjustment system is designed with the flexibility to incorporate higher-order functions, allowing future governance to adjust the system to a power of N if needed. The ability of the protocol to self-adjust under normal market conditions reinforces confidence in its real-world performance and lays the foundation for handling more complex scenarios in future testing phases.

8.1.3. Test Phase 2: Stress Testing

Oceanpoint's dynamic rate system was further stress-tested through four simulations, each designed to assess its ability to maintain stability under extreme market conditions. These tests introduced sudden shocks, such as rapid inflows and outflows of stablecoins and BSPTs, to observe

how the system adjusted its treasury composition and collateralization ratios. By analyzing its response to volatile scenarios, the evaluation aimed to validate the protocol's resilience and ability to self-correct in unpredictable market environments.

Stress-test Type	Stress-test Description
Hyper-demand for POINT	A large influx of stablecoins triggers significant POINT minting.
Large stablecoin withdrawal	A major outflow of stablecoins forces a contraction in POINT supply.
BSPT treasury influx	A sudden deposit of BSPTs increases treasury holdings, prompting additional POINT minting.
Large BSPT withdrawal	A rapid outflow of BSPTs reduces treasury collateral, requiring a balancing response.

Stress-testing parameters	
Initial BSPT Ratio	50%
Target BSPT Ratio	50%
Steps	100
POINT Circulating supply	1,000,000 POINT
TVL Value	1,000,000 USD
Steps of Shock	50
Size of Shock	50%

Results & Insights

Overall, our Phase II stress tests confirm that the smart contract's dynamic rate system is robust under extreme conditions. In the stablecoin influx test, the treasury composition converged to within $\pm 1\text{--}2\%$ of the target with collateralization consistently above 100%, while the withdrawal test showed that even significant reductions in treasury value maintained strong backing.

Similarly, the BSPT influx test demonstrated that when users sell BSPTs in exchange for POINT tokens, both treasury value and POINT circulation rise proportionally—keeping collateralization near

100%—and the BSPT outflow test confirmed that reducing BSPT holdings and burning POINT tokens restores balance effectively. Minor deviations in convergence suggest that further fine-tuning of the nudge mechanism might be beneficial, but no major failures were detected across the scenarios.

8.1.4. Test Phase 3: Multi-Event Stress Simulation

Oceanpoint's ability to withstand complex, unpredictable market conditions was tested through a large-scale simulation incorporating all four major stress events. These included hyper-demand for POINT, large stablecoin withdrawals, BSPT treasury inflows, and large BSPT withdrawals.

Unlike previous tests, stress events listed below were injected at random intervals to mimic real-world uncertainty, evaluating how the protocol's dynamic rate system adapts in multi-event scenarios.

Stress-test Type	Stress-test Description
Hyper-demand for POINT	A large influx of stablecoins triggers significant POINT minting.
Large stablecoin withdrawal	A major outflow of stablecoins forces a contraction in POINT supply.
BSPT treasury influx	A sudden deposit of BSPTs increases treasury holdings, prompting additional POINT minting.
Large BSPT withdrawal	A rapid outflow of BSPTs reduces treasury collateral, requiring a balancing response.

Results & Insights

The phase III stress simulation appears to be functioning largely as expected. Across the random stress events—including hyper-demand for POINT, large stablecoin withdrawals, BSPT treasury influx, and BSPT withdrawals—the dynamic rate adjustments are activated and drive the BSPT ratio in the expected direction.

In scenarios where both treasury value and POINT circulation are adjusted proportionally (as in the BSPT influx and withdrawal events), the collateralization ratio remains stable at 100%. The system's gradual nudge toward the target ratio is evident after each shock, and the dynamic parameters (NAV discount, premium, deposit, and withdrawal taxes) adjust smoothly in line with deviations.

While there may be some minor oscillations immediately following stress events, overall the simulation confirms that the smart contract logic is robust under complex, multi-event conditions.

8.1.5. Summary

Simulations and stress tests have been integral in validating the robustness of Oceanpoint's Dynamic Rate System.

Under normal market conditions (Phase I), baseline simulations—spanning various initial and target BSPT ratios—demonstrated that the system's cubic, non-linear adjustments in NAV discounts, premiums, and stablecoin taxes reliably steered the treasury composition toward the desired balance (e.g., 80/20) while maintaining collateralization at or above 100% relative to POINT's circulating supply.

In Phase II, discrete stress events were simulated, including hyper-demand for POINT (large stablecoin influx), large stablecoin withdrawals, BSPT treasury influx, and BSPT withdrawals; each scenario confirmed that dynamic rate adjustments quickly restored equilibrium, with collateralization remaining strong even under extreme conditions.

Phase III further challenged the protocol by subjecting it to continuous, randomly occurring stress events, and the results showed that despite minor oscillations immediately after shocks, the system consistently converged back toward the target ratio and preserved robust collateral backing.

Collectively, these simulations provide compelling evidence that the Oceanpoint smart contract logic can manage both treasury composition and collateralization effectively, ensuring resilient performance in live market environments.

9. Conclusion & Outlook

Oceanpoint v1.0 marks the beginning of a new financial layer within the Blocksquare ecosystem — one that brings together tokenized real estate and decentralized liquidity in a way that is secure, modular, and scalable. Over the course of this whitepaper, we've introduced the architecture, tokenomics, and phased roadmap that underpin Oceanpoint's function as the liquidity engine for tokenized real estate.

We began by identifying the fragmentation and illiquidity of property tokens (BSPTs) as a critical bottleneck to scaling real estate tokenization. We then introduced Oceanpoint's core innovation: the POINT aggregator token, which enables seamless swaps between BSPTs and stablecoins, while embedding flatcoin-like mechanics designed to preserve or grow purchasing power. As detailed in the system architecture, Oceanpoint's smart contracts and token flows are engineered to adapt dynamically to changing market conditions, with safeguards such as NAV-based premiums and discounts, overcollateralization, and the critical "POINT Zero" backstop mechanism.

To ensure real-world alignment and resilience, Oceanpoint's early test phases served as both sandbox and springboard — from community-driven staking programs to real estate-backed asset pools and marketplace launchpads. These components provided tangible proof of concept while laying the foundations for what is to come.

A full suite of pool mechanisms now supports liquidity across property-backed tokens, governance assets (BST), and crypto-native trading pairs. Importantly, these modules were designed not as short-term liquidity mining programs, but as perpetual, cash-flow-sustained systems — allowing for alignment with the long-duration nature of real estate itself.

Governance has been carefully approached through a multi-signature council with applications open to new members, forming a transparent structure that can progressively decentralize over time. BST token holders will eventually play a central role in protocol upgrades, rate adjustments, and the addition of new collateral types. Governance is not merely a component of Oceanpoint — it is the protocol's compass.

Looking ahead, Oceanpoint's roadmap paves the way toward BSPT-based lending, advanced asset management, and automated loan repayments using real-world yield. Each phase strengthens the role of real estate within DeFi while reinforcing the POINT token's position as a liquidity and collateral aggregator.

Oceanpoint and Blocksquare together offer more than infrastructure — they offer a path forward. A path where real estate becomes programmable, capital-efficient, and universally accessible. Where

liquidity doesn't stop at borders. And where the stability and yield of physical assets flow into the heart of the decentralized economy.

The foundation has been laid. The engine is coming online. And the future of real estate finance is being built — block by block.